

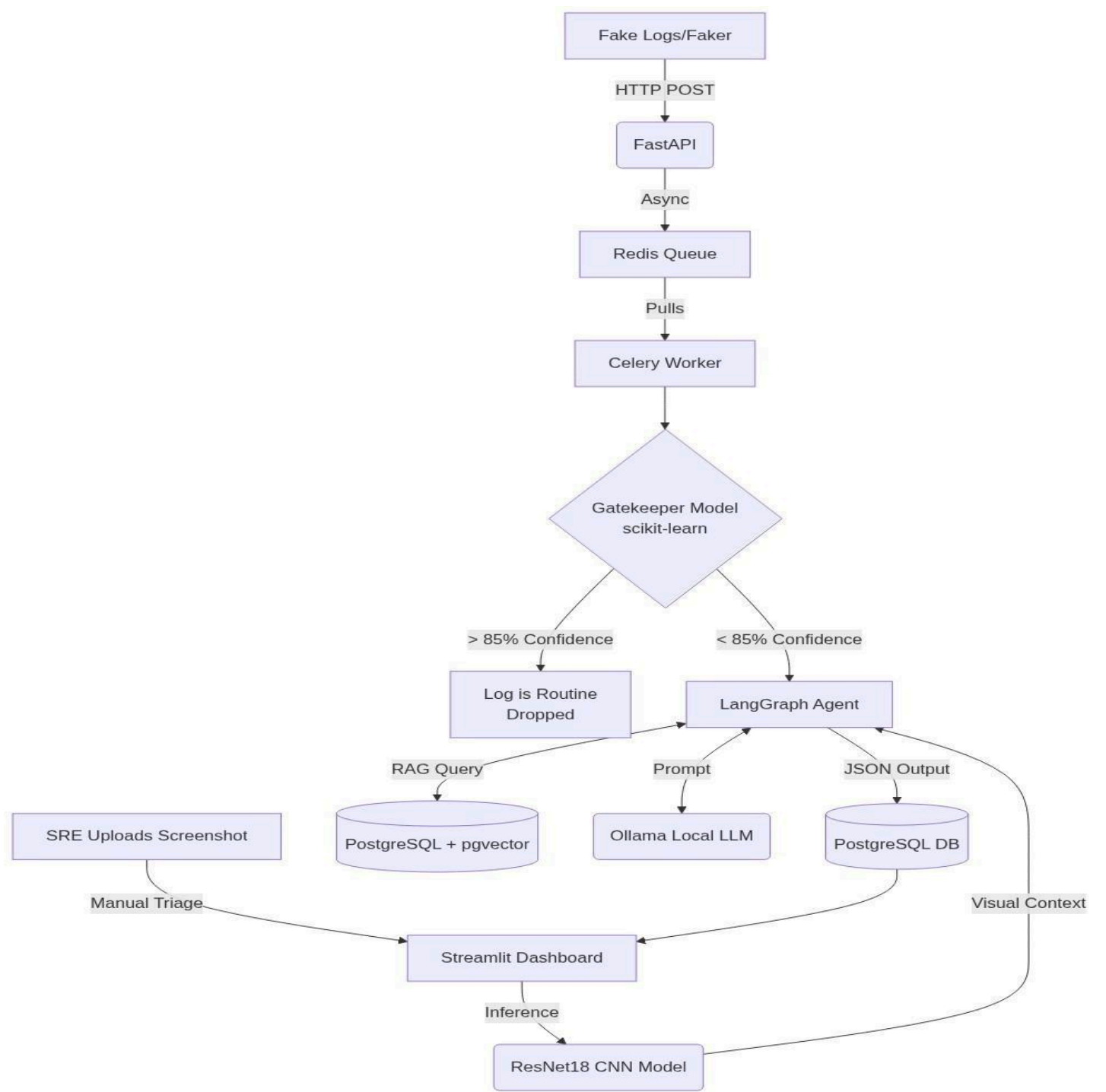
Approach Document: Agentic SRE Log Triage Pipeline

■ The Objective

In large software systems, thousands of log messages are generated every minute. Most of these are "routine" (e.g., successful logins, minor connection retries). However, a few are critical anomalies indicating that the system is crashing or behaving unpredictably. Traditionally, human Site Reliability Engineers (SREs) have to manually sift through these logs, try to understand the error, search through old documentation for similar past issues, and figure out how to fix them. This is slow and expensive. The goal of this project is to build an automated, intelligent pipeline that: 1. **Triages cheap, routine errors instantly** without wasting expensive computing resources. 2. **Unleashes an AI "Agent" to automatically research and document complex, novel outages**, mimicking what a human engineer would do. 3. **Runs entirely locally and for free**, saving money by avoiding costly cloud AI providers for every single log message.

■ Complete Project Architecture

Below is the high-level architecture diagram detailing how data flows from the initial log generation through the various machine learning and orchestrator components, and finally to the dashboard.



■ The Technology Stack

Think of this project as a factory assembly line. Each piece of technology is a specific machine responsible for one job.

Layer	Selected Tool	Purpose in Architecture (What it does)
Log Generator	Faker (Python)	Simulates traffic. It creates a continuous stream of fake logs (both normal ones and rare crashes) so we can test the system.
The Front Door	FastAPI	The web server. It receives the stream of logs from our applications and acts as the entry point to our pipeline.
The Waiting Room	Redis + Celery	A super-fast queue. If 10,000 logs arrive at once, FastAPI quickly throws them into this queue (Redis). Celery workers pick them up one by one so the system never crashes from overload.
The Gatekeeper	scikit-learn (Machine Learning)	A very fast, traditional ML model. It looks at a log and asks, "Have I seen this a million times before?" If yes, it drops it. If no, it rings the alarm.
The Orchestrator	LangGraph	The workflow manager. It tells the AI how to solve a problem step-by-step (e.g., "First read the log, then search the database, then write a fix").
The AI Brain	Ollama (Running Local LLMs)	The actual artificial intelligence (like a local ChatGPT). It runs directly on your machine to analyze the complex errors without sending sensitive company data to the cloud.
The Knowledge Base	PostgreSQL + pgvector	The company's memory. It stores standard logs AND semantic embeddings of past incidents for the AI to search.
The Quality Control	Instructor / Pydantic	Forces the AI brain to output strict, organized data (like a filled-out form) instead of chatty conversational text.
The Business UI	Streamlit	A sleek dashboard for humans to see how much money the AI has saved and view the AI's analysis of crashes in real-time.
System Monitoring	Prometheus + Grafana	Monitors the health of the factory itself (e.g., "Is the queue too full?", "Is the AI taking too long to think?").

1. Data Generation: Test Logs and Images

To properly test the system without a live production environment, the project generates artificial data for both text logs and dashboard images.

- **Text Log Generation:** The system uses the Python [Faker](#) library ([tests/faker_stream.py](#)) to simulate traffic. It dynamically

constructs fake logs by generating random UUIDs, URIs, IPv4 addresses, and words. A probability split injects these fake details into standard routine log templates 95% of the time, while static "novel" exception logs are generated 5% of the time to intentionally trigger the AI agent.

- **Image Data Generation:** For the computer vision pipeline, a custom script (`dataset_generator.py`) procedurally generates synthetic time-series arrays representing Grafana charts. Each sequence contains exactly 100 data points.
- **Image Criteria:** A "Healthy" baseline is established between 20 and 40 units, with environmental noise simulated via a Gaussian distribution (mean of 0, standard deviation of 2). To create anomalies, mathematical deviations are injected at a random index between 10 and 80, lasting for 5 to 15 data points. Positive spikes add a value of 40 to 60, while negative drops subtract up to 15. All values are clipped between 0 and 100 to maintain realism. The data is plotted against a dark `#1e1e2e` background with cyan lines (axes removed) and split 80/20 for training and validation.

2. Data Preprocessing Pipelines

Before the machine learning models can understand the data, it must be mathematically formatted.

- **Text Preprocessing:** Raw logs are passed through a pipeline using scikit-learn's `TfidfVectorizer`. It filters out low-value words using `stop_words='english'`. It then converts the text into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF), assigning higher mathematical importance to rare, unique error words.
- **Image Preprocessing:** Uploaded dashboard screenshots are processed using PyTorch's `torchvision.transforms`. They are forcefully resized to exactly 224x224 pixels to match the mathematical input shape required by the CNN. The RGB pixel values are normalized by subtracting the ImageNet mean (`[0.485, 0.456, 0.406]`) and dividing by the standard deviation (`[0.229, 0.224, 0.225]`), which centers colors around zero. During training, random horizontal flips are applied as data augmentation to prevent overfitting.

3. Asynchronous Queueing: Redis and Celery

To prevent the main FastAPI web server from crashing during heavy traffic spikes, the project implements a queueing system.

- FastAPI receives incoming logs and immediately pushes them into a Redis queue. Redis serves as a fast, in-memory message broker deployed via a lightweight Alpine container on port 6379.
- A Celery worker application (`sre_worker`) continuously connects to the Redis broker and pulls these tasks asynchronously one by one, ensuring data is processed at a sustainable pace.

4. Machine Learning Classification

The project utilizes two distinct models to classify incoming data.

- **Text Classification (ML Model):** The text "Gatekeeper" uses a **Logistic Regression** binary classifier provided by scikit-learn. It analyzes the TF-IDF vectorized logs to determine if they are "routine" (class 1) or "novel" (class 0). If the model's confidence that a log is routine falls below the 85% threshold, it flags the log as an anomaly and triggers the LangGraph agent.
- **Image Classification (CNN Model):** The manual visual triage pipeline relies on **ResNet18**, a Convolutional Neural Network implemented in PyTorch. Through transfer learning, its final fully-connected layer is modified into a custom classification head to output a binary classification: "Healthy" or "Anomalous".

5. AI Orchestration, Local LLM, and Slack Alerts

When an anomaly is detected, the workflow is handed over to the AI layer.

- **LangGraph:** This tool orchestrates the investigation workflow through four primary nodes: `extract`, `retrieve`, `analyze_visuals`, and `synthesize`.
- **Ollama:** Ollama runs entirely locally, acting as the system's "AI Brain" to process complex errors without sending sensitive data to cloud providers. During the final synthesis node, the AI is prompted with the raw log, visual context, and historical data to determine a root cause and suggest a fix.
- **Instructor & Slack Alerts:** The `Instructor` library forces Ollama to output strict JSON data matching a Pydantic schema instead of conversational text. One required field is a `slack_alert`. This alert is simply a drafted text message generated by the AI. It is not configured to send to live webhooks; rather, it is stored in the database and displayed on the UI for human review.
- **AI Cost Usage:** Because Ollama runs locally on your machine, it avoids costly cloud AI provider fees per request. The Streamlit dashboard visually calculates and tracks this "Cost Avoidance" to show users how much money the local AI has saved.

6. Databases and RAG Pipeline

The project utilizes PostgreSQL for two entirely different memory functions, enhancing the AI's capabilities via Retrieval-Augmented Generation (RAG).

- **Traditional PostgreSQL:** Stores structured, transactional data such as the log message, timestamps, model confidence scores, and the AI's final JSON output (root cause, fix, and Slack alert). This allows the dashboard to query exact keywords via standard SQL.
- **pgvector:** The PostgreSQL database is enhanced with the `pgvector` extension to store semantic vector embeddings of historical company post-mortems. Ollama generates these numerical representations using the `nomic-embed-text` model.
- **RAG (Retrieval-Augmented Generation):** RAG prevents the AI from hallucinating by grounding its answers in historical

company data. When a new log arrives, LangGraph generates a query embedding and performs a semantic similarity search in pgvector using Cosine Distance.

- The system finds the top 2 historical chunks closest in mathematical meaning to the new error, feeds them to Ollama, and asks it to base its fix solely on that history.

7. Monitoring: MLflow, Prometheus and Grafana

To monitor the health of this entire triage "factory," the system utilizes standard observability tools.

- **MLflow (Experiment & Trace Tracking):** MLflow is integrated into the Celery workers. For every execution, it tracks the input log parameters, whether an image was provided, the structured entities extracted, and the final JSON synthesis outputs. This helps in auditing what the AI decided and tracking prompt/model performance.
- **Prometheus:** Deployed via Docker Compose, it scrapes system metrics (like queue fullness or AI inference time) from the FastAPI server every 15 seconds.
- **Grafana:** Connected to Prometheus, it exposes port 3000 to provide a web UI that translates these raw time-series metrics into real-time visual dashboards. (Note: While Grafana monitors system health, the resulting charts are what the CNN is trained to analyze when uploaded manually to the UI).

8. User Interface and Deployment Outputs

1. **Streamlit UI:** The system's final output layer is a Streamlit dashboard. It provides a sleek UI where human SREs can upload manual dashboard screenshots, view real-time log anomaly alerts, read the AI's structured JSON analyses (root causes and fixes), review the drafted Slack messages, and track local cost savings.
2. **Docker Containerization:** The entire backing service architecture is orchestrated via a `docker-compose.yml` file. It configures essential images including `redis:7-alpine`, `ankane/pgvector:latest`, `prom/prometheus:v2.45.0`, and `grafana/grafana:10.0.3`. Docker Compose maps the necessary ports to localhost (e.g., 5432 for Postgres, 6379 for Redis) and establishes persistent storage volumes like `postgres_data` so data survives container restarts. Total there are 4 containers.

9. CI/CD Pipeline (GitHub Actions)

The deployment lifecycle is fully automated using GitHub Actions via `.github/workflows/ci-cd.yml` on push to the main branch.

1. **Code Quality & Linting:** Runs pylint and performs a SonarCloud static code analysis to ensure code standards.
2. **Automated Testing:** Runs pytest against dummy endpoints and logic blocks to prevent regression.
3. **Build & Push:** Uses a multi-stage Dockerfile to build the required microservices and pushes the resulting image to the GitHub Container Registry (GHCR).
4. **Continuous Deployment:** Deploys the latest stable image to Azure Web App via the `azure/webapps-deploy` action.

■ The Multimodal Hybrid RAG Pipeline (Retrieval-Augmented Generation)

When the AI encounters a completely new error, we don't want it to just guess or hallucinate an answer. We use a strategy called **RAG**.

The Purpose of RAG

RAG grounds the AI's intelligence in your company's actual historical data. It retrieves specific, relevant facts from your database and feeds them to the AI before asking it to answer a question.

When the AI encounters a completely new error, we don't want it to just guess or hallucinate an answer. We use a strategy called RAG. This pipeline is Multimodal, meaning it can accept and process both text logs and image screenshots (e.g. from Grafana dashboards).

The RAG Workflow

1. **The Stored Corpus:** Over time, engineers write incident reports (Post-Mortems). These documents form the corpus of knowledge.
2. **Chunking & Vectorization:** Post-mortems are split into smaller, focused chunks.
 - * **Dense (Semantic) Vectors:** Each chunk is passed through Ollama to generate a numerical embedding, saved in a vector column in pgvector. The embedding model used is `nomic-embed-text`.
 - * **Sparse (Keyword) Vectors:** The text is also converted into a tsvector in PostgreSQL for fast full-text keyword searches using a GIN index.
3. **Hybrid Retrieval Strategy:** When a query is made, we use two separate retrievers:
 - * **Dense Retriever:** Uses Cosine Distance in pgvector to find semantically similar chunks.
 - * **Sparse Retriever:** Uses PostgreSQL `plainto_tsquery` to find exact keyword matches.
4. **Re-Ranking:** The results from both retrievers are combined and deduplicated. They are then scored and re-ranked using the cross-encoder/`ms-marco-MiniLM-L-6-v2` model (via `sentence-transformers`). The top K chunks with the highest reranker scores are kept.
5. **Generation & Multimodal Synthesis:** The AI is given a prompt containing the text error, the top historical post-mortems from the reranker, and visual context generated from any uploaded screenshots processed by the CNN model. Based on this text and visual history, the AI formulates the root cause and suggested fix.

$$\text{Cosine Similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

$$\text{Cosine Distance} = 1 - \text{Cosine Similarity}$$

Image dataset generation

1. Dataset Array Configuration:

- **Sequence Length:** Each generated time-series sequence is comprised of exactly 100 individual data points.

2. Normal "Steady State" Parameters:

- **Starting Level:** A random baseline is established between 20 and 40 units, mimicking standard operational metrics like a server idling at 30% CPU load.
- **Environmental Noise:** Natural fluctuations are simulated by applying a Gaussian distribution (mean=0, std_dev=2) to every point in the sequence.

3. Artificial Anomaly Trigger Logic: To simulate system failures, the script injects mathematical deviations based on these rules:

- **Onset Timing:** Deviations are programmed to appear at a random index between 10 and 80, ensuring the anomaly is fully visible within the frame.
- **Event Duration:** The simulated incident spans a random window of 5 to 15 data points.
- **Spike Model:** For positive anomalies, a surge value between 40 and 60 is added to the baseline, representing a traffic burst.
- **Drop Model:** Negative anomalies subtract a value between 15 and the current base_level, mimicking a service blackout or crash.

4. System Boundary Controls:

- **Data Clipping:** To maintain realism, np.clip forces all values into a 0 to 100 range, preventing impossible metrics like negative utilization.

5. Folder Distribution Logic:

- **Validation Strategy:** A 0.2 split is enforced; 80% of images populate the train/ directory, while 20% are reserved in the val/ folder for unbiased testing.
- The generator produces a balanced set of exactly 100 samples per category (Healthy vs. Anomalous) by default.

Image Pre-Processing

Before a screenshot can be analyzed by the neural network, it must be standardized through a pre-processing pipeline using PyTorch's torchvision.transforms: 1. **Resizing:** Chart images come in various resolutions. All input images are resized to a standard dimension of **224x224 pixels**, which is the expected input size for our architecture. 2. **Color Normalization:** The RGB channels of the image are normalized using the ImageNet standard mean ([0.485, 0.456, 0.406]) and standard deviation ([0.229, 0.224, 0.225]). This helps the model converge faster during training by keeping pixel values within a stable range. 3. **Data Augmentation (During Training):** To prevent overfitting on specific dashboard themes, we apply random horizontal flips. This makes the model more robust to varying UI layouts while maintaining the core shape of the data spikes.

CNN Training and Architecture

The core of the computer vision module is a Convolutional Neural Network (CNN). - **Transfer Learning:** We leverage a pre-trained **ResNet18** model. Since training a deep network from scratch requires massive amounts of data, using ResNet18 (pre-trained on millions of generic images) provides a strong foundational understanding of shapes, lines, and edges. - **Custom Classification Head:** We modify the final fully connected layer of the ResNet18 model to output binary classification probabilities: **Anomalous vs. Healthy**. - **Optimization:** The model is trained using the **Adam optimizer** with a learning rate of 0.001 and **Cross-Entropy Loss**. We freeze the early feature-extraction layers and only compute gradients for the final layers to speed up training and prevent catastrophic forgetting.

Evaluation Metrics

To ensure the model is reliable for production incident response, we evaluate it on a hold-out validation set using the following metrics: - **Accuracy:** The overall percentage of correctly classified charts. - **Precision:** Of all the charts the model flagged as

anomalous, how many were *actually* anomalous? (Minimizing false alarms). - **Recall:** Of all the *actual* anomalies, how many did the model catch? (Minimizing missed outages). - *Note:* In SRE, **Recall** is generally prioritized over Precision, as missing a critical outage spike is much worse than investigating a false alarm.

Challenges Faced

Integrating Computer Vision into an IT operations context presented several unique challenges:

1. **Data Scarcity for Training / Dataset Generation:** Unlike standard objects (cats, dogs, cars), there is no massive open-source dataset of "anomalous Grafana charts." We had to write a custom script (`dataset_generator.py`) to procedurally generate our own dataset. - **Time-Series Simulation:** The script uses numpy to generate base metric levels (e.g., CPU hovering between 20-40%) and adds Gaussian noise to simulate realistic fluctuations. - **Injecting Anomalies:** For the "anomalous" class, the script programmatically injects sudden mathematical spikes (adding +40 to +60 to the base level) or drops (subtracting -15) at random intervals within the time-series array. - **Rendering:** Using matplotlib, these arrays are plotted against a dark '#1e1e2e' background (mimicking Grafana's default dark theme) with neon cyan lines. - **Exporting:** The axes and labels are removed so the CNN focuses purely on the shape of the data. The script loops this process to instantly generate hundreds of perfectly labeled .png images into train/ and val/ directories, completely bypassing the need for manual data collection.
2. **Dashboard Theme Variability:** Real-world dashboards come in light mode, dark mode, and use various color palettes. The model initially struggled with these variations. We addressed this by ensuring our synthetic training data mimicked various background themes and focusing the model on the *shape* of the line rather than the color.
3. **Computational Overhead:** Running CNN inference alongside local LLM generation (Ollama) can be extremely memory-intensive on a local machine. We optimized this by choosing the lightweight ResNet18 architecture over larger models (like ResNet50 or VGG16) to ensure the agent remains fast and responsive without requiring a massive GPU.